

Chapter 9. Scaling Node

Scaling an application is about making it able to handle more work without slowing down or crashing. You can scale any application by either giving its servers more memory or CPU power, or by adding more servers.

For Node, with its non-blocking and event-driven model, scaling is something that's baked into the core of the runtime environment. A Node application should be composed of multiple small, distributed nodes. You can run the same Node process on multiple CPU cores (or multiple servers) and then load balance the requests among them. Node has a built-in module to help with that.

In this chapter, we'll learn all about the Node `node:cluster` module, which can help improve the workload performance of a Node process by making it utilize the full CPU power of a server. It also improves the availability and uptime of servers.

Strategies of Scalability

While making an application able to handle more work is the most popular reason to scale them, there are more reasons. Applications are also scaled to increase their availability and their tolerance to failure.

If you're tasked with scaling an existing application, your options are either to scale it *vertically* by giving its servers more power (memory and CPU) or *horizontally* by adding more servers. However, if you're thinking of scaling as you build your applications (which you should be), you need to understand the following three strategies and make scalability choices based on that understanding and the applications' projected use over time:

Cloning

The easiest thing to do to scale a big application is to clone it multiple times and have each cloned instance handle part of the workload (with a load balancer, for example). This does not cost a lot in terms of development time, and it's highly effective. This strategy is the minimum you should do, and Node has the built-in `node:cluster` module to make it easier for you to implement the cloning strategy on a single server.

Decomposing

We can also scale an application by decomposing it based on functionalities and services. This means having multiple applications with different code bases and sometimes with their own dedicated databases and UIs. This strategy is commonly associated with the term *microservice*, where *micro* indicates that those services should be as small as possible (in reality, the size of the service is not what's important but rather the enforcement of loose coupling and high cohesion between services). The implementation of this strategy is often not easy and could result in long-term unexpected problems, but when done right the advantages are great.

Splitting

We can also split the application into multiple instances where each instance is responsible for only one part of the application's data. This strategy is often named *horizontal partitioning*, or *sharding* in databases. Data partitioning requires a lookup step before each operation to determine which instance of the application to use. For example, maybe we want to partition our users based on their country or language. We need to do a lookup of that information first.

Applications might pick one or more of these strategies based on their current and projected needs.

The Cluster Module

Node's `node:cluster` module can be used to run multiple instances of the same Node process on different CPU cores and balance the working load among them. It uses the `child_process.fork` method under the hood to give us an easy way to fork an application process as many times as we need. On one server machine, we can fork a process as many times as there are CPU cores on that machine. The `node:cluster` module can then take over the management of these forks, load balance all requests coming to the application across all forked processes, and reload any forks when needed.

The `node:cluster` module is Node's helper for us to implement the cloning scalability strategy vertically on one server. When you have a big server with a lot of resources or when it's easier and cheaper to add more resources to one server rather than adding new servers, the `node:cluster` module is a great option for a really quick implementation of the cloning strategy.

Even small servers usually have multiple cores, and even if you're not worried about the load on your Node server, you should use the `node:cluster` module anyway to increase your server availability and fault tolerance. It's a simple step with big advantages.

Taking advantage of what the `node:cluster` module has to offer is simple. You create a *primary* process that forks a number of *worker* processes and manages them. Each worker process represents an instance of the application. All incoming requests are handled by the primary process, which is the one that decides which worker process should handle an incoming request.

The primary process's job is simple. It actually just uses a round-robin algorithm to pick a worker process. This is enabled by default on most platforms, but it can be modified to let the load balancing be handled differently (for example, by the OS or with custom logic).

The round-robin algorithm distributes the load evenly across all available worker processes on a rotational basis. The first request is forwarded to the first worker process, the second to the next worker process, and so on. When the end of the worker process list is reached, the algorithm starts again from the beginning.

Round-robin is one of the simplest load-balancing algorithms, but there are several other algorithms that can be used for load balancing:

Least connections

The primary process sends an incoming request to the worker with the fewest active connections. This helps distribute the workload more evenly.

Weighted round-robin

Workers are assigned different weights, and those with higher weights handle more incoming requests.

Random

The primary process picks a worker randomly to handle an incoming request.

IP hash

The primary process uses the IP of the client making a request to determine which worker should handle that request.

URL hash

The primary process uses the requested URL to determine which workers should handle that request.

Each routing algorithm has its own use cases, benefits, and disadvantages. Picking an algorithm depends on the requirements of the application and the environments in which it runs.

Primary and Worker Processes

To see the `node:cluster` module in action, let's use this basic HTTP server. Create a *slow-server.js* file for this code:

```
import { createServer } from 'node:http';

createServer((req, res) => {
  for (let i = 0; i < 1e8; i++) {
    // Simulate CPU work:
  }

  res.end();
}).listen(3000, () => {
  console.log(`Process ${process.pid}`);
});
```

Note how I added an empty `for` loop to simulate some CPU work before responding.

With the `node:cluster` module, we'll be working with multiple processes, so I'm logging the process ID (with `process.pid`) when the HTTP server starts, to see what processes are being created.

Before we create a cluster for this server and fork many workers for it, let's do a simple *load testing* for it. Load testing is basically sending many requests for the server and reporting how much load it was able to handle.

There are many tools you can use to perform load testing, from basic ones like ApacheBench (AB) to fully featured ones like Artillery. To keep things simple for our example, we'll use the simple `loadtest` npm package.

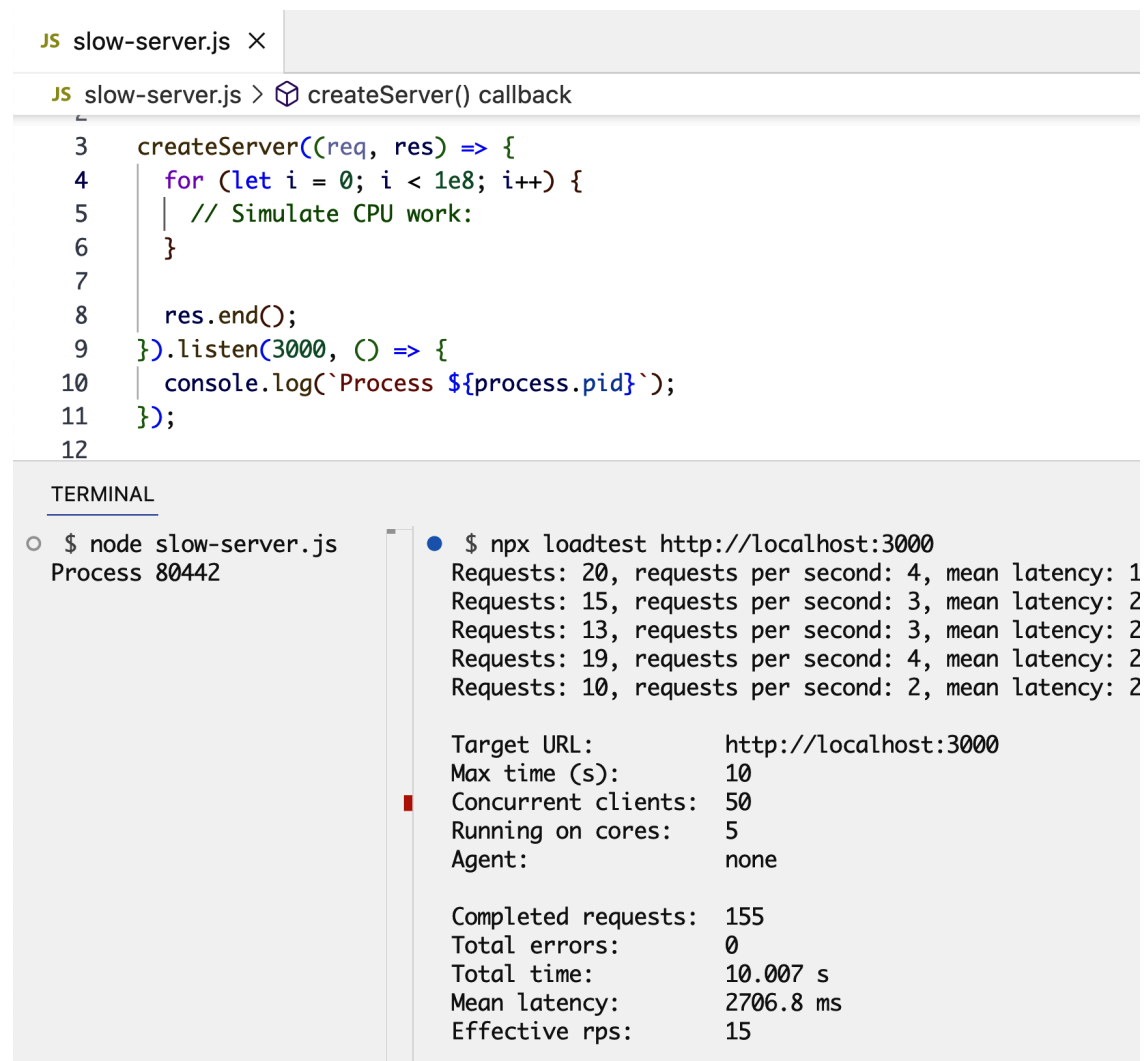
After installing the package, run the HTTP server and then run the `loadtest` command (in a different terminal) to perform the load testing:

```
$ node slow-server.js
```

```
$ npx loadtest http://localhost:3000/
```

[Figure 9-1](#) shows the performance summary I got when I did the testing on my computer.

The single-node server was able to handle about 15 requests per second on my computer. This result will be different on different computers.



```
JS slow-server.js ×
JS slow-server.js > createServer() callback
3  createServer((req, res) => {
4    for (let i = 0; i < 1e8; i++) {
5      // Simulate CPU work:
6    }
7
8    res.end();
9  }).listen(3000, () => {
10    console.log(`Process ${process.pid}`);
11  });
12

TERMINAL
○ $ node slow-server.js
  Process 80442
● $ npx loadtest http://localhost:3000
  Requests: 20, requests per second: 4, mean latency: 1
  Requests: 15, requests per second: 3, mean latency: 2
  Requests: 13, requests per second: 3, mean latency: 2
  Requests: 19, requests per second: 4, mean latency: 2
  Requests: 10, requests per second: 2, mean latency: 2

  Target URL:      http://localhost:3000
  Max time (s):    10
  Concurrent clients: 50
  Running on cores: 5
  Agent:           none

  Completed requests: 155
  Total errors:       0
  Total time:        10.007 s
  Mean latency:      2706.8 ms
  Effective rps:     15
```

Figure 9-1. Performance summary for the single-node server

Now that we have a reference benchmark for a single node, let's see what difference the `node:cluster` module will make for this performance.

Here's the code (added to a `cluster.js` file) we need to create a primary process with the `node:cluster` module and make that process fork many worker processes to run the same HTTP server:

```

import cluster from 'node:cluster';
import os from 'node:os';

if (cluster.isPrimary) {
  const cpus = os.availableParallelism();

  console.log(`Forking for ${cpus} CPUs`);
  for (let i = 0; i < cpus; i++) {
    cluster.fork();
  }
} else {
  import('./slow-server.js');
}

```

We first imported both the `node:cluster` module and the `node:os` module. We used the `node:os` module to figure out how many parallel processes we can run using the `availableParallelism` function. Depending on the server where the code is run, this function will return a number between one and the maximum number of CPU cores available on that server (`os.cpus().length`). The plan is to fork one worker process for each available CPU core.

The `node:cluster` module gives us the handy Boolean flag `isPrimary` to determine if this `cluster.js` file is being loaded as a primary process. The first time we execute this file, we will be executing the primary process, and that `isPrimary` flag will be set to `true`. In this case, we can instruct the primary process to fork our server as many times as we have CPU cores using the `cluster.fork` method with a loop over the number of CPUs the server has. This makes the cluster take advantage of all the available processing power.

When the `cluster.fork` line is executed from the primary process, the current file, `cluster.js`, is run again, but this time in *worker mode* with the `isPrimary` flag set to `false`.

TIP

There is actually another flag set to `true` in this case (the `isWorker` flag) if you need to use it.

When the application runs as a worker, it can start doing the actual work. This is where we need to define our server logic, which, for this example, we can do by importing the *slow-server.js* file that we have already.

That's basically it. That's how easy it is to take advantage of all the processing power in a server. To test the cluster, run the *cluster.js* file:

```
$ node cluster.js
Forking for 10 CPUs
Process 15601
Process 15602
Process 15606
Process 15607
Process 15604
Process 15605
Process 15603
Process 15609
Process 15610
Process 15608
```

The computer I tested with had 10 cores, so the `for` loop forked 10 worker processes. It's important to understand that these forks are completely different Node processes. Each forked worker process will have its own memory space and event loop.

When this server gets multiple requests, thanks to the built-in load balancing of the `node:cluster` module, the requests will be handled by different worker processes with different process IDs. The workers will not be exactly rotated in sequence because the `node:cluster` module performs some optimizations when picking the next worker, but the load will be somehow distributed among the different worker processes.

Now, let's use the same `loadtest` command to measure the performance of this cluster, as shown in [Figure 9-2](#).

```
JS slow-server.js × ... JS cluster.js ×
```

```
JS slow-server.js > listen() callback
3  createServer((req, res) => {
4    for (let i = 0; i < 1e8; i++) {
5      // Simulate CPU work:
6    }
7
8    res.end();
9  }).listen(3000, () => {
10   console.log(`Process ${process.pid} is running`);
11 });
12
```

```
JS cluster.js > ...
4  if (cluster.isPrimary) {
5    const cpus = os.availableCpus();
6
7    console.log(`Forking for ${cpus} CPUs`);
8    for (let i = 0; i < cpus; i++) {
9      cluster.fork();
10   }
11 } else {
12   import('./slow-server.js');
13 }
```

TERMINAL

```
○ $ node cluster.js
Forking for 10 CPUs
Process 87097
Process 87090
Process 87099
Process 87095
Process 87092
Process 87093
Process 87094
Process 87091
Process 87096
Process 87098
```

Requests: 130, requests per second: 26, mean latency: 372
Requests: 130, requests per second: 26, mean latency: 366
Requests: 126, requests per second: 25, mean latency: 383
Requests: 126, requests per second: 25, mean latency: 382
Requests: 120, requests per second: 24, mean latency: 389
Requests: 259, requests per second: 26, mean latency: 391

Target URL:	http://localhost:3000
Max time (s):	10
Concurrent clients:	50
Running on cores:	5
Agent:	none

Completed requests:	1270
Total errors:	0
Total time:	10.017 s
Mean latency:	385.3 ms
Effective rps:	127

Figure 9-2. Performance summary for the server cluster

The same computer that handled only 16 requests per second with a single process is now able to handle 123 requests per second! This is a more than sevenfold increase in performance, and all we had to do to make that happen was add a few simple lines of code.

NOTE

We didn't have to run the worker processes on different ports. The `node:cluster` module takes care of making the worker processes share ports.

Broadcasting Messages

Communicating between the primary process and the workers is simple because under the hood the `node:cluster` module is using the `child_process.fork` API, which means we also have IPC channels available between the primary process and each worker process.

We can access the list of worker objects using `cluster.workers`, which is an object that holds a reference to all workers and can be used to read information about these workers. Since the communication channels between the primary process and all workers are already established, to broadcast a message to all workers we can use a simple loop over them and use the `send` method on each:

```
Object.values(cluster.workers).forEach(worker => {  
  worker.send(`Hello Worker ${worker.id}`);  
});
```

The `Object.values` method is used here to get an array of all workers from the `cluster.workers` object. Then, for each worker, we use the `send` function to send over any value we want. In a worker file—`slow-server.js`, in our example—to read a message received from this primary process, we can register a handler for the `message` event on the `process` object. Here's an example:

```
process.on('message', msg => {  
  console.log(`Message from primary: ${msg}`);  
});
```

[Figure 9-3](#) shows what I see when I test these two additions to the `cluster.js/slow-server.js` example. I added a `setTimeout` around the sending to make sure all the forking is done before sending.

```
JS slow-server.js × ... JS cluster.js ×
JS slow-server.js > ... JS cluster.js > setTimeout() callback > forEach() ca
9   }).listen(3000, () => {
10   |   console.log(`Process $
11   | });
12
13   process.on('message', (m
14   |   console.log(`Message f
15   | });
16
8   for (let i = 0; i < cpus; i++) {
9   |   cluster.fork();
10  | }
11
12  setTimeout(() => {
13  |   Object.values(cluster.workers).forEach
14  |   |   worker.send(`Hello Worker ${worker.i
15  |   });
16  | }, 1_000);
```

TERMINAL

```
o $ node cluster.js
Forking for 10 CPUs
Process 87278
Process 87274
Process 87277
Process 87279
Process 87276
Process 87272
Process 87273
Process 87275
Process 87280
Process 87281
Message from primary: Hello Worker 2
Message from primary: Hello Worker 1
Message from primary: Hello Worker 3
Message from primary: Hello Worker 7
Message from primary: Hello Worker 6
Message from primary: Hello Worker 4
Message from primary: Hello Worker 5
Message from primary: Hello Worker 10
Message from primary: Hello Worker 8
Message from primary: Hello Worker 9
|
```

Figure 9-3. Messages between primary and worker processes

Every worker process received a message from the primary process.

Let's make this communication example a bit more practical. Let's say we need our HTTP service to reply with the number of users we have in the database. To keep things simple, we can use a fake function to replace the database user count logic:

```
// Fake double
const dbUsersCount = (() => {
  let count = 1;
  return () => {
    count = 2 * count;
    return count;
  };
});
```

```
};  
})();
```

This function simply returns `2` the first time it's called, then doubles its returned value for each subsequent call.

This is just for simplified testing, but since the real scenario here is that the function will make a database request, we need to avoid making multiple requests by the multiple workers we have. We can use some sort of caching in the worker processes and update the cache after a certain period of time (say 60 seconds). This is better, but the 10 workers would still be doing 10 database requests every 60 seconds.

Wouldn't it be great if another process did the database requests and shared the count with all workers every 60 seconds? This is where we start combining the cloning strategy with the decomposing strategy!

For simplicity, let's make the primary process carry out the request to count users in the database (ideally, another entity should do that). We'll make the call to `dbUsersCount` in `cluster.js` and then use a `for` loop to broadcast the count value to all workers every 60 seconds.

Here's how I changed the `cluster.js` file to implement that:

```
// In the cluster.isPrimary branch:  
  
const dbUsersCount = (() => {  
  let count = 1;  
  return () => {  
    count = 2 * count;  
    return count;  
  };  
})();  
  
const updateWorkers = () => {  
  const usersCount = dbUsersCount();  
  Object.values(cluster.workers).forEach((worker) => {  
    worker.send({ usersCount });  
  });  
});
```

```
};

updateWorkers();
setInterval(updateWorkers, 60_000);
```

Here we're invoking `updateWorkers` for the first time and then invoking it every 60 seconds using a `setInterval`. This way, every 60 seconds, all workers will receive the new user count value over the IPC channel, and only one database connection will be made every 60 seconds.

In the server code, we can use the `usersCount` value in the `message` event handler. We can simply store that value in a top-scope variable and use it anywhere in that module.

Here's how I changed *slow-server.js* to implement that:

```
let usersCount;

createServer((req, res) => {
  for (let i = 0; i < 1e8; i++) {
    // Simulate CPU work:
  };

  res.write(`Process ${process.pid}\n`);
  res.end(`Users: ${usersCount}`);
}).listen(3000, () => {
  console.log(`Process ${process.pid}`);
});

process.on('message', (msg) => {
  usersCount = msg.usersCount;
});
```

I changed the server response to include the `usersCount` value. If you test the cluster code now, during the first 60 seconds you'll get `2` as the users count from all workers (and only one database request will be made). After 60 seconds, all workers would start reporting the new user count, `4` (and only one other database request would be made).

This is all possible thanks to the communication channels between the primary process and all workers.

Increasing Availability

One of the problems in running a single instance of a Node application is that when that instance crashes, it has to be restarted. This causes some server downtime between these two actions, even if the process was monitored and a restart is automated.

This also applies to the case when the server has to be restarted to deploy new code. With one instance, there will be server downtime that affects the availability of the system.

When we have multiple instances, the availability of the system can be easily increased with only a few extra lines of code. Let's make that happen.

First, to make the example practical for testing, let's simulate a random crash in the server process. We can do a `process.exit` call inside a timer that fires after a random amount of time:

```
// In slow-server.js

setTimeout(() => {
  process.exit(1);
}, Math.random() * 10_000);
```

When a worker process exits like this, the primary process will be notified using the `exit` event on the `node:cluster` object. We can register a handler for that event and simply fork a new worker process to replace the crashed process:

```
// In the cluster.isPrimary branch:

cluster.on('exit', (worker) => {
  if (worker.exitedAfterDisconnect === true) return;
```

```
    console.log(
      `Worker ${worker.id} crashed.` +
      'Starting a new worker...',
    );

    cluster.fork();
  });
```

Note how I added an `if` condition to make sure the worker process actually crashed and wasn't manually disconnected and killed by the primary process itself. For example, the primary process might decide that we are using too many resources based on the load patterns it sees, and it will need to kill a few workers in that case. To do so, we can use the `disconnect` methods on any `worker` object and, in that case, the `exitedAfterDisconnect` flag will be set to `true`, so the `if` statement will guard against forking a new worker for that case.

If we run `cluster.js` with the preceding handler (and the random crash in `slow-server.js`), after a random number of seconds, workers will start to crash, and the primary process will immediately fork new workers to increase the availability of the system. You can actually measure the availability using the same `loadtest` command and see how many requests the server will not be able to handle overall (because some of the unlucky requests will still have to face the crash case).

[Figure 9-4](#) shows the output of `loadtest` on my computer while the random crash was happening.

```
JS slow-server.js × JS cluster.js ×

JS slow-server.js > ...
6   }
7
8   res.end();
9 }).listen(3000, () => {
10  console.log(`Process ${process.pid} started`);
11 });
12
13 setTimeout(() => {
14   process.exit(1);
15 }, Math.random() * 10_000);
16

JS cluster.js > ...
12 cluster.on('exit', (worker) => {
13   if (worker.exitedAfterDisconnect) return;
14
15   console.log(
16     `Worker ${worker.id} crashed.
17     Starting a new worker...`,
18   );
19   cluster.fork();
20 }
21 });
22 } else {
23   import('./slow-server.js');
24 }
```

TERMINAL

```
Worker 9 crashed.Starting a new worker...
Worker 4 crashed.Starting a new worker...
Process 88344
Worker 8 crashed.Starting a new worker...
Process 88347
Process 88348
Worker 12 crashed.Starting a new worker...
Process 88350
Worker 15 crashed.Starting a new worker...
Process 88354
Worker 10 crashed.Starting a new worker...
Process 88355
Worker 17 crashed.Starting a new worker...
Process 88356
```

```
$ npx loadtest http://localhost:3000

Target URL:      http://localhost:3000
Max time (s):    10
Concurrent clients: 50
Running on cores: 5
Agent:           none

Completed requests: 1228
Total errors:      16
Total time:        10.003 s
Mean latency:      327.7 ms
Effective rps:     123
```

Figure 9-4. Performance summary while the server is randomly crashing

Only 16 requests failed out of 1,228. That’s over 98% availability. By adding a few simple lines of code, we now don’t have to worry about process crashes anymore. The primary process will keep an eye on the failed processes for us and replace them on demand.

Zero-Downtime Restarts

When we deploy new code to production servers, all Node processes need to be restarted. During a restart, requests to these processes will fail.

In a cluster, instead of restarting all worker processes together, we can restart them one at a time. This way, while one worker is being restarted, the others will continue to serve requests, and we won’t have any server downtime at all.

Implementing this with the `node:cluster` module is actually really easy. Let me show you how.

First, since we don't want to restart the primary process once it's up (because that would be total downtime for all workers), we need a way to send the primary process a command to instruct it to start restarting its workers (one by one).

There are a few ways to do that depending on what OS the server is running. With Linux systems, we can listen to a process signal like `SIGUSR1` or `SIGUSR2`. These signals can be sent to a running process by issuing a `kill` command like this:

```
$ kill -SIGUSR2 PID
```

In Node, we can listen to this `SIGUSR2` signal as an event on the `process` object, and within its handler we can do whatever we want:

```
process.on('SIGUSR2', () => {  
  // Do something  
});
```

WARNING

Don't use `SIGUSR1`. Node uses that for debugging purposes. Also, on Windows, these process signals are not supported, and you would have to find another way to instruct the primary process to do something. There are some alternatives. For example, you can use standard input or socket input. You can also monitor the existence of a `process.pid` file and track any `remove` events on it.

Now that we know how to instruct the primary process to restart its workers, we can have it do so by disconnecting each worker and forking a new one to replace it. However, since we want it to disconnect one worker at a time, we need to have it disconnect one worker, wait for that, fork another worker, wait for the fork to start taking requests, then disconnect the next worker.

A worker process has an `exit` event similar to the `node:cluster` event but specific to the worker, and it means that specific worker has exited. We can use that `exit` event to start the forking of the next worker.

A worker process also has a `listening` event that is triggered after a call to `listen` happens in that worker. We can use that event to start disconnecting the next worker.

Putting it all together, here are the changes I made to *cluster.js* to implement the gradual reloading of all workers:

```
// In the cluster.isPrimary branch:

console.log(
  `To restart workers, use: kill -SIGUSR2 ${process.pid}`,
);

process.on('SIGUSR2', () => {
  const workers = Object.values(cluster.workers);

  const restartWorker = (workerIndex) => {
    const worker = workers[workerIndex];
    if (!worker) return;

    worker.on('exit', () => {
      if (worker.exitedAfterDisconnect === false) return;

      console.log(`Exited process ${worker.process.pid}`);

      cluster.fork().on('listening', () => {
        restartWorker(workerIndex + 1);
      });
    });

    worker.disconnect();
  };

  restartWorker(0);
});
```

The `workers` array has the list of all cluster workers. The `restartWorker` function receives the index of a worker to be restarted, starting from `0`. This way, we can do the restarting in sequence by having this function call itself with the next index value when it's ready for the next worker.

In the `restartWorker` function, we got a reference to the worker to be restarted. Since we will be calling this function recursively to form a sequence, we need a stop condition. When we no longer have a worker to restart, we can simply return. We then disconnect that worker (using `worker.disconnect`), but before restarting the next worker we need to fork a new one to replace this current disconnecting one. We can use the `exit` event on the current worker to fork a new worker when the current one exists.

We have to make sure that the `exit` event was actually triggered after a normal disconnect call (the opposite of the crash example). We can use the `exitedAfterDisconnect` flag. If this flag is `false`, the exit was caused by something other than a disconnect call, and in that case, we should return and do nothing. But if the flag is `true`, we can go ahead and fork a new worker to replace the one that we're disconnecting.

When the new forked worker is ready, which is determined here using the `listening` event, we can restart the next one by calling the `restartWorker` function again with the next index value for the `workers` array.

That's all we need for a zero-downtime restart. To test it, I added a `console.log` statement to give us the ID of the cluster process.

Running the logged `kill -SIGUSR2 PID` command should start the workers' reloading process.

To see the zero-downtime in action, before running the kill command, run the same `loadtest` command. [Figure 9-5](#) shows the results when I tested this.

```
JS slow-server.js × JS cluster.js ×

JS slow-server.js > ...
1 import { createServer } from 'http'
2
3 createServer((req, res) => {
4   for (let i = 0; i < 1e8; i++) {
5     // Simulate CPU work:
6   }
7   res.end();
8 }).listen(3000, () => {
9   console.log(`Process ${process.pid} started`);
10 });
11
12

JS cluster.js > ...
8 for (let i = 0; i < cpus; i++) {
9   cluster.fork();
10 }
11
12 console.log(
13   `To restart workers, use: kill -SIGUSR2 88792`
14 );
15
16 process.on('SIGUSR2', () => {
17   const workers = Object.values(cluster.workers);
18   const restartWorker = (workerId) => {
19     const worker = workers[workerId];
20     worker.kill('SIGUSR2');
```

TERMINAL

```
○ $ node cluster.js
Forking for 10 CPUs
To restart workers, use: kill -SIGUSR2 88792
Process 88794
Process 88797
Process 88795
Process 88796
Process 88800
Process 88793
Process 88799
Process 88798
Process 88801
Process 88802
Exited process 88793
Process 88839
Exited process 88794
Process 88840
Exited process 88795
Process 88843
Exited process 88796

● $ npx loadtest http://localhost:3000
Requests: 121, requests per second
Requests: 126, requests per second
Requests: 122, requests per second
Requests: 120, requests per second
Requests: 122, requests per second
Requests: 243, requests per second

Target URL:      http://localhost:3000
Max time (s):    10
Concurrent clients: 50
Running on cores: 5
Agent:           none

Completed requests: 1235
Total errors:      0
Total time:        10.004 s
Mean latency:      395.6 ms
Effective rps:     123
```

Figure 9-5. Performance summary while the workers are restarted

No requests failed at all during the reloading of this cluster!

Handling State

One of the important challenges to understand in a scaled environment is how to handle in-memory state and any stateful communications in general.

Since the workers have their own separate memory spaces, we cannot cache things in memory in one worker. The other workers will not have access to that cached value.

If you need to cache things in a scaled environment, you have to use a separate entity and read/write to that entity's API from all workers. This entity can be a database server, a service like Redis, or a dedicated Node process with a read/write API for all other workers to communicate with.

This is known as *distributed caching*, and it's an example of the decomposing scalability strategy for an application. This is something that you should do even if you're running your application on a single core.

Another example of state management is handling user authentication and managing user sessions. In a scaled environment, a request for authentication comes to the primary process and gets sent to one worker. That worker will start recognizing the state of the user. However, when the same user makes another request, the load balancer will eventually send them to other workers, which do not have them as authenticated. Keeping a reference to an authenticated user session in one instance is not going to work anymore. Instead, we need to keep workers stateless and manage stateful information in an external entity.

Another way to deal with the user sessions problem is to use what's known as *sticky sessions*. It's not as efficient as using an external stateful entity, but it's certainly easier to implement. We can simply use a routing algorithm like IP hash.

Sticky sessions is a simple example of the splitting scaling strategy. When a user authenticates with a worker process, a record of that relation is kept in the primary process. Then, when the same user sends a new request, we use that record to keep sending them to the same worker process. This way, the code on the server side does not have to be changed.

While a routing algorithm like IP hash is not natively supported by Node's `node:cluster` module, implementing it is fairly simple. We can put the workers in an array, reduce the IP of an incoming request into an index within that array, and have the worker with that index handle the request. Here's a very basic implementation of that:

```
import os from 'node:os';
import { fork } from 'node:child_process';
import { createServer } from 'node:http';

const cpus = os.availableParallelism();
const workers = [];

for (let i = 0; i < cpus; i++) {
  workers.push(fork('./slow-server.js'));
}

function getWorkerIndex(ip) {
  const hash = ip.split('.').reduce(
    // Hash IP octets and combine them
    (hash, part) => Number(part) + 256 * hash,
  );
  return hash % cpus;
}

createServer((req, res) => {
  const ip = req.socket.remoteAddress;
  const workerIndex = getWorkerIndex(ip);
  workers[workerIndex].send({ req, res });
  // IPC logic
}).listen(3000);
```

While using a sticky load-balancing algorithm like IP hash is simple to implement and can be suitable and efficient for some cases (like a small application with a few workers), it's certainly not ideal. It can lead to uneven workload distribution, and it doesn't scale very well. Keeping Node workers stateless and managing state with an external service is a better approach in terms of workload distribution and scalability.

TIP

Sticky load-balancing algorithms like IP hash can be easily configured and used in other load balancers, like nginx, and many others.

Process Managers

If you don't want to manage your own cluster code, you can use one of the advanced process managers packages available for Node. These packages wrap the `node:cluster` module and provide a CLI to simplify the management of forked processes.

One of the popular advanced process managers for Node is PM2, and while most of its features cost money to use, it offers the basics of running and managing a cluster of workers for free.

Install the `pm2` package with `npm` and take a look at its help page (with a `-h` option). It has many commands and options to start, monitor, and manage a cluster (along with many other features).

For example, to launch a cluster of workers for a Node process, you just start the Node process with the `start` command, and use the `-i` option (with either a custom number of forks, or `max` to use the maximum possible forks):

```
$ npx pm2 start script.js -i max
```

To perform a zero-downtime restart, you can use the `reload` command:

```
$ npx pm2 reload all
```

To see a list of all worker processes with their status and usage, you can use the `list` command:

```
$ npx pm2 list
```

Summary

Scaling in Node is a first-class concept. It's why Node is named Node, after all.

There are many scaling strategies like cloning, decomposing, and splitting. The focus of this chapter was mainly on the cloning strategy.

Node's built-in `node:cluster` module can be used to create a primary process that forks many worker processes and manages them. It can start and stop them, send them data, and receive data from them. Workers can share ports, and the primary process can distribute the workload among them to increase the performance of the server.

The workload can be distributed among worker processes in multiple ways. The `node:cluster` module uses a simple cyclical routing algorithm to load balance incoming requests among workers, but other routing algorithms can be used. Caching and managing stateful communication in general can be done by using other routing algorithms or by using external services.

The primary process can also monitor the workers, replace faulty ones immediately, and restart workers one by one. This improves the uptime and fault tolerance of the server.

In the next chapter, we'll explore some of the tools that can be used to improve development workflows and help with building, testing, deploying, and maintaining Node code.